

INFORME DE ACTIVIDAD PRÁCTICA

Sistema seguro de registro de pacientes en Python

Desarrollo de una aplicación de consola con validación, privacidad, integridad de archivos y pruebas de seguridad.

Estudiante

Carrera

Analista Programador

Asignatura

Docente

Sección

Fecha

20 de julio de 2026

Este proyecto es solamente académico. Se usaron datos ficticios y no debe utilizarse para guardar información médica real.

1. Objetivo y alcance

El objetivo fue construir un sistema de consola en Python que permita registrar pacientes ficticios de forma controlada. El programa valida y sanitiza los datos antes de guardarlos, crea un identificador seudónimo, muestra reportes anonimizados y comprueba la integridad del archivo JSON mediante SHA-256. También se aplicaron pruebas automatizadas, cobertura y análisis estático con Bandit.

El alcance es educativo. La solución usa recursos básicos: módulos de la biblioteca estándar, archivos JSON, funciones pequeñas y un menú de texto. No pretende reemplazar un sistema clínico real.

Actividad	Resultado
Aplicación y datos ficticios	Completada
Pruebas automatizadas	25 pruebas aprobadas
Cobertura	59 % total
Bandit	0 hallazgos
SonarQube	Configurado; no ejecutado por falta de Docker

2. Organización del proyecto

Archivo	Responsabilidad
app.py	Menú principal, registro, búsqueda y reporte.
validaciones.py	Validación del RUT, edad, correo y sanitización de textos.
privacidad.py	Identificador seudónimo y vistas anonimizadas.
almacenamiento.py	Lectura y escritura JSON, escritura atómica y SHA-256.
tests/	Pruebas de validación, privacidad e integridad.

Separar las responsabilidades hizo más fácil revisar cada parte y evita que toda la lógica quede mezclada en un solo archivo.

3. Matriz de riesgos

Riesgo	Impacto	Probabilidad	Control aplicado	CIA
Ingreso de caracteres no permitidos	Datos incorrectos o contenido malicioso.	Media	Sanitización con expresiones regulares y límites de longitud.	I
Edad negativa, decimal o fuera de rango	Registro inconsistente.	Media	Conversión controlada y rango de 0 a 120 años.	I
Exposición de RUT, nombre, correo o diagnóstico	Pérdida de privacidad.	Alta	Reporte anonimizado y exclusión completa del diagnóstico.	C
Uso del RUT como identificador público	Facilita relacionar el registro con una persona.	Media	ID aleatorio generado con secrets.	C
Modificación manual del archivo JSON	Datos alterados sin autorización.	Media	SHA-256, compare_digest y cierre preventivo.	I

Riesgo	Impacto	Probabilidad	Control aplicado	CIA
Archivo inexistente o JSON corrupto	El programa no puede iniciar.	Media	Creación automática y manejo de excepciones.	D
Dependencia maliciosa o nombre parecido	Ejecución de código no confiable.	Baja	Paquetes conocidos, nombres fijados y fuentes oficiales.	C/I/D
Código sugerido por IA sin revisión	Errores o vulnerabilidades no detectadas.	Media	Comprensión, pruebas, Bandit y revisión humana.	C/I/D

C = confidencialidad, I = integridad y D = disponibilidad. La prioridad principal fue evitar la exposición de información y detectar cambios del archivo.

4. Aplicación de OWASP SAMM

Función simplificada	Aplicación en el proyecto
Gobierno	Reglas: no usar datos reales, no incluir secretos y revisar IA.
Diseño	Matriz de riesgos, módulos separados y datos sensibles identificados.
Implementación	Validaciones, sanitización, errores controlados e ID seguro.
Verificación	Pruebas manuales, pytest, cobertura y Bandit.
Operaciones	Carpeta automática, escritura atómica y detección de alteraciones.

Yo entiendo SAMM como una forma de ordenar y mejorar las prácticas de seguridad. En este caso el nivel es inicial: existen controles y evidencia, pero todavía no hay automatización continua ni métricas históricas.

5. Aplicación de Microsoft SDL

Etapas	Trabajo realizado
Capacitación	Revisión de CIA, privacidad, IA responsable y dependencias.
Requisitos	Registro, búsqueda, reporte, integridad y rechazo de entradas inválidas.
Diseño	Arquitectura modular y riesgos definidos antes de programar.
Implementación	Funciones pequeñas, secrets, SHA-256 y manejo de excepciones.
Verificación	25 pruebas, 59 % de cobertura y 0 hallazgos Bandit.
Liberación	Sin datos generados, secretos ni entorno virtual en la entrega.
Respuesta	Alerta crítica y cierre cuando el JSON no coincide con su hash.

6. Aplicación de la tríada CIA

Principio	Cómo se aplicó
Confidencialidad	RUT, nombre y correo se ocultan; el diagnóstico no aparece en reportes y se usa un ID seudónimo.
Integridad	Los datos se validan, se sanitizan, se escriben de forma atómica y el JSON se protege con un resumen SHA-256.
Disponibilidad	Se crean carpetas automáticamente, se controlan errores y el menú continúa después de entradas inválidas.

7. Evidencias de ejecución

Se registraron tres pacientes completamente ficticios. Las fechas, los identificadores y los resultados siguientes corresponden a una ejecución real del programa en el entorno de verificación.

```
Evidencia - registro

REGISTRO DE TRES PACIENTES FICTICIOS
=====
Paciente 1: Paciente registrado correctamente.
Identificador asignado: PAC-0JHTECCGA9KF
Paciente 2: Paciente registrado correctamente.
Identificador asignado: PAC-R4AGJH6WMCPM
Paciente 3: Paciente registrado correctamente.
Identificador asignado: PAC-P2D7PII3CXRI

Los identificadores cambian en cada ejecución por usar secrets.
```

Figura 1. Registros e identificadores generados con secrets.

```
Evidencia - reporte anonimizado

REPORTE ANONIMIZADO
=====

Paciente número 1
-----
ID: PAC-0JHTECCGA9KF
Nombre: A** P*****
RUT: *****-5
Edad: 31
Correo: a*****@example.com
Fecha de registro: 2026-07-21T02:46:28.363996+00:00

Paciente número 2
-----
ID: PAC-R4AGJH6WMCPM
Nombre: B*** E*****
RUT: *****-1
Edad: 44
Correo: b*****@example.com
Fecha de registro: 2026-07-21T02:46:28.395246+00:00

Paciente número 3
-----
ID: PAC-P2D7PII3CXRI
Nombre: C*** D***
RUT: *****-2
Edad: 26
Correo: c*****@example.com
Fecha de registro: 2026-07-21T02:46:28.410875+00:00
```

Figura 2. Reporte donde no aparece el diagnóstico y los datos están ocultos.

```
Evidencia - JSON y SHA-256

ARCHIVO datos/pacientes.json
=====
[
  {
    "id_paciente": "PAC-0JHTECCGA9KF",
    "nombre": "Ana Prueba",
    "rut": "12345678-5",
    "edad": 31,
    "correo": "ana.prueba@example.com",
    "diagnostico": "Control preventivo",
    "fecha_registro": "2026-07-21T02:46:28.363996+00:00"
  },
  {
    "id_paciente": "PAC-R4AGJH6WMCPM",
    "nombre": "Bruno Ejemplo",
    "rut": "11111111-1",
    "edad": 44,
    "correo": "bruno.ejemplo@example.com",
    "diagnostico": "Revisión general",
    "fecha_registro": "2026-07-21T02:46:28.395246+00:00"
  },
  {
    "id_paciente": "PAC-P2D7PII3CXRI",
    "nombre": "Carla Demo",
    "rut": "22222222-2",
    "edad": 26,
    "correo": "carla.demo@example.com",
    "diagnostico": "Evaluación de rutina",
    "fecha_registro": "2026-07-21T02:46:28.410875+00:00"
  }
]

ARCHIVO datos/pacientes.sha256
=====
2f6ad234dead6cfe5f7b3b0758f8fd2098646a060a1d9b28b6ce0a35cc156543
```

Figura 3. Fragmento del JSON ficticio y resumen SHA-256 generado.

8. Pruebas negativas

Se intentaron los seis casos solicitados. Todos fueron rechazados mediante ValueError y el programa pudo continuar.

```
Evidencia - pruebas negativas

PRUEBAS NEGATIVAS
=====
[RECHAZADO] Edad negativa: La edad debe contener solo números enteros.
[RECHAZADO] Edad decimal: La edad debe ser un número entero.
[RECHAZADO] RUT inválido: El RUT ingresado no es válido.
[RECHAZADO] Correo inválido: El correo electrónico no tiene un formato válido.
[RECHAZADO] Nombre vacío: El nombre no puede estar vacío.
[RECHAZADO] Diagnóstico demasiado largo: El diagnóstico no puede superar 150 caracteres.
```

Figura 4. Resultado de las entradas inválidas.

9. Integridad del archivo

Después de guardar los pacientes se cambió una letra del JSON de prueba sin actualizar el archivo SHA-256. En el siguiente inicio el sistema detectó la diferencia y se cerró para no trabajar con información posiblemente alterada.

Evidencia - archivo alterado

```
=====
SISTEMA SEGURO DE REGISTRO DE PACIENTES
Aplicación educativa: no utilizar datos reales
=====

ALERTA CRÍTICA
La integridad de los datos no pudo comprobarse. El archivo podría haber sido modificado.
La aplicación se cerrará para evitar trabajar con datos posiblemente alterados.

Código de salida: 0
```

Figura 5. Alerta crítica después de modificar el JSON.

10. Pruebas automatizadas y cobertura

Se ejecutaron 25 pruebas con pytest. Incluyen los casos indicados en la guía, una prueba adicional para el largo del diagnóstico y dos pruebas para guardar, cargar y detectar una alteración. Todas finalizaron correctamente.

Evidencia - pytest

```
cachedir: .pytest_cache
collecting ... collected 25 items
tests/test_almacenamiento.py::test_guardar_y_cargar_pacientes PASSED [ 4%]
tests/test_almacenamiento.py::test_detectar_archivo_modificado PASSED [ 8%]
tests/test_privacidad.py::test_generar_id_paciente PASSED [ 12%]
tests/test_privacidad.py::test_ids_deben_ser_distintos PASSED [ 16%]
tests/test_privacidad.py::test_generar_id_rechaza_longitud_invalida PASSED [ 20%]
tests/test_privacidad.py::test_anonimizar_nombre PASSED [ 24%]
tests/test_privacidad.py::test_anonimizar_rut PASSED [ 28%]
tests/test_privacidad.py::test_anonimizar_correo PASSED [ 32%]
tests/test_privacidad.py::test_vista_no_incluye_diagnostico PASSED [ 36%]
tests/test_validaciones.py::test_sanitizar_nombre_valido PASSED [ 40%]
tests/test_validaciones.py::test_sanitizar_nombre_elimina_numeros PASSED [ 44%]
tests/test_validaciones.py::test_sanitizar_nombre_vacio_genera_error PASSED [ 48%]
tests/test_validaciones.py::test_validar_edad_correcta PASSED [ 52%]
tests/test_validaciones.py::test_validar_edad_negativa_genera_error PASSED [ 56%]
tests/test_validaciones.py::test_validar_edad_excesiva_genera_error PASSED [ 60%]
tests/test_validaciones.py::test_validar_edad_decimal_genera_error PASSED [ 64%]
tests/test_validaciones.py::test_validar_correo_correcto PASSED [ 68%]
tests/test_validaciones.py::test_validar_correo_incorrecto PASSED [ 72%]
tests/test_validaciones.py::test_calcular_digito_verificador PASSED [ 76%]
tests/test_validaciones.py::test_validar_rut_correcto PASSED [ 80%]
tests/test_validaciones.py::test_validar_rut_incorrecto PASSED [ 84%]
tests/test_validaciones.py::test_sanitizar_diagnostico_elimina_html PASSED [ 88%]
tests/test_validaciones.py::test_sanitizar_estructura_completa PASSED [ 92%]
tests/test_validaciones.py::test_sanitizar_estructura_con_campo_faltante PASSED [ 96%]
tests/test_validaciones.py::test_diagnostico_demasiado_largo_genera_error PASSED [100%]
===== 25 passed in 0.32s =====
```

Figura 6. Pruebas automatizadas aprobadas.

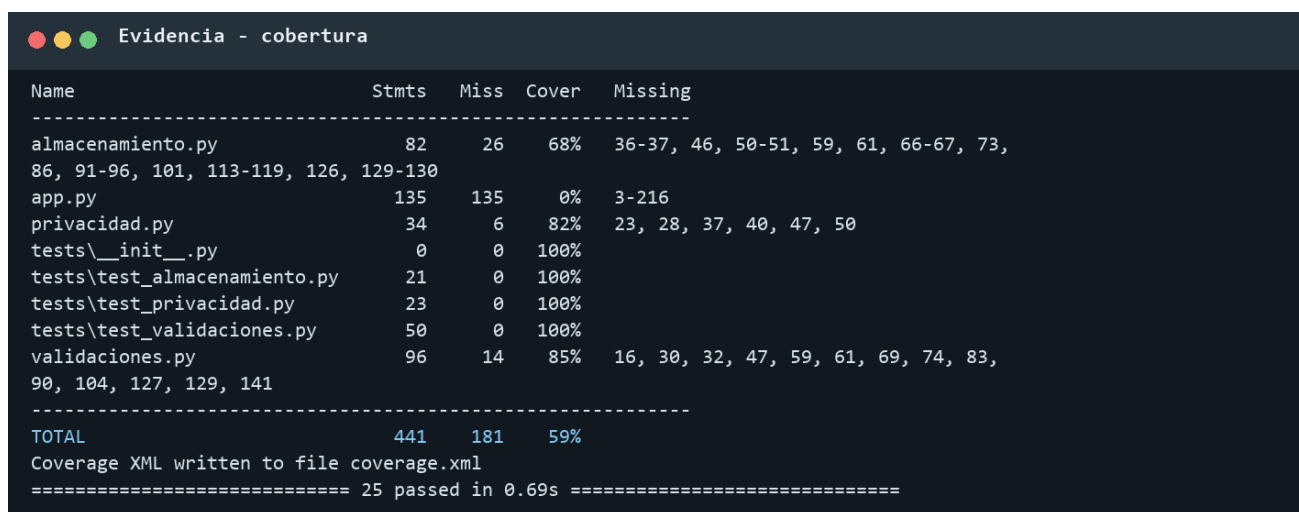


Figura 7. Cobertura total informada: 59 %.

La cobertura no es más alta porque el menú interactivo de app.py fue probado de forma manual y no con pruebas unitarias. Un 100 % tampoco garantizaría que el sistema sea seguro; solamente indica qué líneas fueron ejecutadas.

11. Resultado de Bandit

Dato solicitado	Resultado
Hallazgos	0
Severidad	0 bajos, 0 medios y 0 altos
Confianza	No aplica, porque no hubo hallazgos
Correcciones por Bandit	Ninguna
Falsos positivos	No se observaron

```
Evidencia - Bandit

Run started:2026-07-21 02:46:32.957618+00:00

Test results:
    No issues identified.

Code scanned:
    Total lines of code: 470
    Total lines skipped (#nosec): 0
    Total potential issues skipped due to specifically being disabled (e.g., #nosec
BXXX): 0

Run metrics:
    Total issues (by severity):
        Undefined: 0
        Low: 0
        Medium: 0
        High: 0
    Total issues (by confidence):
        Undefined: 0
        Low: 0
        Medium: 0
        High: 0

Files skipped (0):

[main] INFO    profile include tests: None
[main] INFO    profile exclude tests: None
[main] INFO    cli include tests: None
[main] INFO    cli exclude tests: None
[main] INFO    running on Python 3.11.9
... (evidencia abreviada en el informe)
```

Figura 8. Bandit no identificó problemas en 470 líneas de código.

Bandit no reemplaza las pruebas. Durante la ejecución manual apareció un error que Bandit no marcó: la guía llamaba `compare_digest` desde `hashlib`. La función correcta está en `hmac`, por lo que se cambió a `hmac.compare_digest` y se agregó una prueba de integridad. Esto demuestra la necesidad de revisión humana.

12. Resultado de SonarQube

Métrica	Resultado en este equipo
Bugs	No ejecutado
Vulnerabilidades	No ejecutado
Security Hotspots	No ejecutado
Code Smells	No ejecutado
Cobertura importada	No ejecutado
Duplicación	No ejecutado

Docker Desktop no está instalado en el equipo usado para preparar la entrega, por lo tanto no fue posible levantar SonarQube ni obtener métricas reales. No se inventaron resultados. El archivo `sonar-project.properties` está incluido y `coverage.xml` ya fue generado. Para completar esta evidencia se debe instalar Docker, iniciar SonarQube, crear el proyecto, guardar el token en la variable `SONAR_TOKEN` y ejecutar SonarScanner.

Estado: pendiente de ejecución en un equipo con Docker. Si el docente exige captura del panel, esta sección debe actualizarse antes de entregar.

13. Selección y revisión de dependencias

Dependencia	Uso	Criterio aplicado
Biblioteca estándar de Python	Aplicación	Sin paquetes externos en producción.
pytest 8.4.2	Pruebas	Paquete conocido, nombre exacto y versión fijada.
pytest-cov 7.0.0	Cobertura	Complemento específico para pytest.
Bandit 1.9.4	Análisis estático	Herramienta dedicada a seguridad en Python.

Se usó requirements-lock.txt para registrar también las dependencias transitivas. Antes de instalar un paquete se debe revisar el nombre exacto, mantenedor, documentación, licencia, actividad reciente y vulnerabilidades conocidas, evitando nombres parecidos que puedan ser typosquatting.

14. Registro de uso responsable de IA

Elemento	Registro
Necesidad	Proponer una prueba adicional para el límite del diagnóstico.
Prompt	Propón una prueba pytest para comprobar que sanitizar_diagnostico rechace textos de más de 150 caracteres. No uses datos reales y explica el assert.
Propuesta revisada	with pytest.raises(ValueError): sanitizar_diagnostico('a' * 151)
Decisión	Aceptada porque coincide con LONGITUD_MAXIMA_DIAGNOSTICO, no usa información sensible y la prueba pasó.
Cambios	Se agregó nombre descriptivo, tipo de retorno y comentario de trazabilidad.

Durante la preparación se utilizó Codex como asistente de IA. El prompt es compatible con GitHub Copilot, pero no se afirma haber usado Copilot. Si la evaluación exige específicamente esa herramienta, se debe repetir el mismo ejercicio allí y guardar su captura. No se enviaron datos reales, claves ni tokens.

15. Respuestas a las preguntas de análisis

Pregunta	Respuesta
1. ¿Cuál es la diferencia entre validar y sanitizar?	Validar comprueba si un dato cumple reglas y puede rechazarlo. Sanitizar limpia o transforma el dato para dejar solamente contenido permitido. En el proyecto se hacen las dos cosas antes de guardar.
2. ¿Por qué no debe almacenarse directamente el dato ingresado?	Porque la entrada del usuario no es confiable. Puede venir vacía, fuera de rango, con formato incorrecto o con caracteres que no deberían quedar en el archivo.
3. ¿Por qué secrets es más apropiado que random para generar identificadores?	secrets utiliza una fuente de aleatoriedad apropiada para usos de seguridad y sus resultados son menos predecibles. random está pensado principalmente para simulaciones y no para identificadores sensibles.
4. ¿La seudonimización elimina todos los riesgos de privacidad?	No. Reemplaza el identificador directo, pero si existe información adicional o una tabla que permita relacionar el seudónimo, todavía se podría identificar a la persona.
5. ¿Ocultar parte del RUT equivale a anonimización irreversible?	No. Es solamente un enmascaramiento visual. El RUT completo sigue almacenado y otros datos podrían permitir volver a identificar al paciente.
6. ¿Qué principio CIA se afecta si un archivo es modificado?	La integridad, porque ya no se puede confiar en que el contenido sea correcto y no haya sido alterado.
7. ¿Qué principio CIA se afecta si el sistema se detiene?	La disponibilidad, porque los usuarios no pueden utilizar el servicio cuando lo necesitan.
8. ¿Qué principio CIA se afecta si se muestra el diagnóstico?	La confidencialidad, debido a que se expone un dato sensible a personas que podrían no estar autorizadas.
9. ¿Por qué no basta con instalar una librería desde PyPI?	PyPI distribuye muchos paquetes, pero igualmente se debe comprobar el nombre, el mantenedor, la documentación, la licencia, las versiones y las vulnerabilidades conocidas.
10. ¿Qué es un ataque de typosquatting?	Consiste en publicar un paquete con un nombre muy parecido al original para aprovechar errores al escribir e instalar código malicioso.
11. ¿Qué diferencia existe entre Bandit y SonarQube?	Bandit revisa patrones de seguridad específicos de código Python. SonarQube analiza calidad general, bugs, vulnerabilidades, hotspots, duplicación y cobertura para varios lenguajes.
12. ¿Qué es un falso positivo?	Es un hallazgo informado por una herramienta que parece un problema, pero después de revisar el contexto se determina que no representa un riesgo real.
13. ¿Por qué un Security Hotspot requiere revisión humana?	Porque la herramienta señala una parte sensible, pero no conoce todo el contexto. Una persona debe decidir si la protección aplicada es suficiente o si debe corregirse.
14. ¿Una cobertura del 100 % garantiza seguridad?	No. Solo indica que las líneas fueron ejecutadas durante las pruebas. Las pruebas podrían tener casos incompletos o no revisar condiciones de seguridad importantes.
15. ¿Por qué no se debe confiar automáticamente en GitHub Copilot?	Porque puede proponer código incorrecto, desactualizado o inseguro. Toda sugerencia debe entenderse, compararse con los requisitos y probarse.
16. ¿Quién es responsable del código generado por IA?	La persona que decide incorporarlo, ejecutarlo y entregarlo. La herramienta no reemplaza la responsabilidad del desarrollador.
17. ¿Qué información nunca debería enviarse a una IA pública?	Datos personales o médicos reales, contraseñas, tokens, claves privadas, código confidencial y cualquier información restringida por la organización.
18. ¿Por qué el token de SonarQube debe protegerse?	Porque permite autenticarse ante el servidor. Si se filtra, otra persona podría ejecutar acciones o acceder a información según los permisos del token.
19. ¿Qué controles faltarían para utilizar el sistema en producción?	Cifrado, gestión de claves, autenticación, roles, permisos, auditoría, respaldos, base de datos transaccional, monitoreo, cumplimiento legal y un plan de incidentes.

Pregunta	Respuesta
20. ¿Cómo ayuda SAMM a mejorar la seguridad de una organización?	Permite evaluar prácticas, definir un nivel actual, priorizar mejoras y avanzar de manera gradual con actividades medibles en gobierno, diseño, implementación, verificación y operaciones.

16. Conclusiones

La actividad permitió comprobar que la seguridad se puede aplicar desde el inicio incluso en un programa sencillo. La validación evitó datos incoherentes, la sanitización limitó caracteres, el reporte redujo la exposición y el hash detectó un cambio manual. La prueba funcional también demostró que una herramienta automática no encuentra todos los errores, porque el problema de `compare_digest` fue detectado al ejecutar el sistema y no por Bandit.

SAMM ayudó a organizar las prácticas y SDL a relacionarlas con cada etapa del desarrollo. El resultado cumple el objetivo académico, pero todavía debe considerarse un prototipo y no un sistema clínico.

17. Limitaciones

- El JSON está en texto claro y no existe cifrado en reposo.
- No hay autenticación, roles ni autorización.
- El hash no usa una clave secreta y está guardado junto al archivo.
- La anonimización es básica y puede no ser irreversible.
- No hay base de datos, auditoría, respaldo automático ni control de concurrencia.
- La cobertura total es 59 % y el menú fue validado manualmente.
- SonarQube quedó pendiente porque Docker no está disponible en este equipo.

18. Propuestas de mejora

- Usar HMAC con una clave guardada fuera del código.
- Cifrar los datos y respaldos, con gestión segura de claves.
- Agregar autenticación, segundo factor, roles y permisos.
- Migrar a SQLite o un servidor de base de datos con consultas parametrizadas.
- Agregar auditoría sin registrar diagnósticos ni identificadores completos.
- Crear respaldos y probar su restauración.
- Automatizar pytest, Bandit y SonarQube en integración continua.
- Aumentar las pruebas del menú y de los casos de error de almacenamiento.

19. Lista de comprobación final

Comprobación	Estado
Programa inicia y menú funciona	Cumplido
Tres pacientes ficticios registrados	Cumplido

Comprobación	Estado
Validación y sanitización	Cumplido
Reporte anonimizado sin diagnóstico	Cumplido
JSON, SHA-256 y detección de cambios	Cumplido
25 pruebas aprobadas	Cumplido
Cobertura XML	Cumplido: 59 %
Bandit JSON	Cumplido: 0 hallazgos
SonarQube	Pendiente: requiere Docker
Dependencias fijadas	Cumplido
Uso de IA documentado	Cumplido
Sin tokens, datos reales ni entorno virtual	Cumplido

Anexo: comandos principales

```
python -m venv venv
.\venv\Scripts\Activate.ps1
python -m pip install -r requirements-dev.txt
python app.py
python -m pytest -v
python -m pytest --cov=. --cov-report=term-missing --cov-report=xml
python -m bandit -r . -x ./venv,./.venv,./tests -f json -o reporte-bandit.json
```

Fuente principal: Guía práctica 'Desarrollo de un sistema seguro de registro de pacientes en Python', material entregado para la actividad.